

Performance Testing Report

Computer Systems and Network Engineering – Final Year
Research Project

Project Title : CargoMate - Smart Robotic Inventory System with Cloud
Integration (Navigation Subsystem)

Student Name : D.K.D.G. Perera

Student ID : IT22312594

Supervisor : Mr. Uditha Dharmakeerthi

Date : 29/04/2026

1. Introduction and Objective

CargoMate is an autonomous warehouse robotic navigation system built for cost-constrained environments, avoiding expensive hardware such as LiDAR or SLAM. The navigation subsystem integrates a five-channel IR sensor array, PN532 RFID module, HC-SR04 ultrasonic sensor, and a Firebase-backed Telegram notification system, all orchestrated by a Finite State Machine (FSM) on an ESP32 microcontroller.

Performance testing evaluates whether the system meets reliability, responsiveness, and operational continuity benchmarks under simulated warehouse conditions. Testing is critical because: (1) delayed obstacle responses risk physical collisions; (2) cloud notification latency directly impacts operator awareness; and (3) sensor polling intervals must remain tight without saturating the ESP32's dual cores. Testing types conducted include functional correctness testing (nine defined scenarios), quantitative response-time testing (obstacle halt and notification latency), sensor accuracy testing (RFID detection rate and IR deviation), and endurance testing (continuous battery operation).

Defined benchmarks: line deviation < 10 mm, obstacle halt < 100 ms, Telegram delivery < 5 s, RFID detection $\geq 98\%$ at nominal speed, line recovery 100% for departures $\leq 45^\circ$, and continuous operation > 1 hour.

2. Performance Testing Environment Setup

Hardware: Testing was conducted on a flat laboratory floor at SLIIT with a black-tape track including straight segments, and three RFID-tagged waypoints (Rack A, B, C). The robot comprised an ESP32 (240 MHz dual-core, 520 KB SRAM), five IR sensors, PN532 RFID (I2C), HC-SR04 ultrasonic sensor, L298N motor driver, two DC motors, and a lithium-ion battery pack with 5V DC-DC buck converter. A smartphone verified Telegram delivery timestamps independently.

Software: Firmware developed in Arduino IDE for ESP32. Libraries: Adafruit PN532 (v1.3.0), FirebaseESP32, UniversalTelegramBot, and ESP32 LEDC PWM at 5 kHz / 8-bit resolution. Backend: Google Firebase Realtime Database. No standard web load-testing tools (JMeter, LoadRunner) were applicable to this embedded context; instead, firmware-level serial port instrumentation and Firebase Console monitoring were used.

Network: 2.4 GHz Wi-Fi with ~ 80 -120 ms round-trip latency to Firebase servers. Tests ran under varied Wi-Fi loads (light and moderate) to stress the notification subsystem. HTTPS cloud calls executed non-blocking on ESP32 Core 0, isolating network latency from motor-critical Core 1 operations.

3. Tools and Frameworks Used

Tool	Purpose	Justification
Serial Port Logger (115200 baud)	Millisecond-level event timestamps	Direct firmware instrumentation; no OS overhead
Firebase Realtime Console	Live FSM state and cloud sync verification	Independent of firmware; validates HTTPS REST calls
Telegram Timestamp Comparison	End-to-end notification latency	Measures true delivery including server push delay
Optical Encoder (GPIO interrupt)	Position estimation via leftPulseCount	Hardware interrupt; zero CPU polling overhead
Digital Stopwatch	Recovery time and endurance measurement	Simple, accurate wall-clock timing for longer events

Standard tools like JMeter were inapplicable as CargoMate has no HTTP server endpoints; workload stress was applied through concurrent subsystem demands (IR polling + ultrasonic polling every 50 ms + non-blocking RFID I2C + Firebase write every 2 s + Telegram dispatch on Core 0) rather than concurrent virtual users.

4. Testing Methodology and Execution

Functional Scenarios: Nine test cases (TC1-TC9) each executed five times. Pass required all five trials to match expected outcomes. Scenarios covered: straight-line following, RFID target detection, reverse navigation, destination RFID detection, obstacle halt, obstacle clearance, power-on motor safety, sensor stabilization delay, and complete FSM state transition validation.

Quantitative Testing: Each metric sampled a minimum of ten times; results expressed as mean $\pm$ SD. Obstacle halt: rigid obstacle placed just under 20 cm across 10 runs. RFID accuracy: four tags $\times$ 10 passes at both nominal (PWM 200/255) and maximum speed = 30 detections per condition. Line recovery: 20 controlled line-loss events split equally between $\leq 45^\circ$ and $> 45^\circ$ departures. Telegram latency: 15 obstacle events across all three rack targets at three positional zones (first-quarter, mid-path, near-destination).

Execution Protocol: Each run began with full power cycle (INIT state) ensuring clean sensor initialization and Firebase connection. Serial log timestamps were zeroed at motor enable. Encoder counts reset each cycle. Obstacle events triggered manually by placing a cardboard barrier. All data recorded trial-by-trial before computing statistics.

Peak Workload Condition: NAVIGATE state active with simultaneous IR polling every loop + ultrasonic poll every 50 ms + RFID I2C non-blocking read + Firebase status write every 2 s + Telegram alert on Core 0. This represents the maximum concurrent subsystem demand.

5. Results and Analysis

Table 1: Quantitative Performance Metrics

Metric	Result	Benchmark	Status
Line deviation - straight segments	< 5 mm	< 10 mm	Exceeds
RFID detection - nominal speed	100% (30/30)	$\geq 98\%$	Exceeds
RFID detection - maximum speed	93.3% (28/30)	$\geq 98\%$	Below
Obstacle halt response time	48 ms $\pm$ 6 ms	< 100 ms	Exceeds
Telegram notification latency	2.3 s $\pm$ 0.7 s	< 5 s	Exceeds
Line recovery - departure $\leq 45^\circ$	100% (20/20)	100%	Meets
Line recovery - departure $> 45^\circ$	80% (16/20)	100%	Below
Mean recovery time (successful)	3.4 s $\pm$ 0.9 s	< 10 s	Exceeds
Continuous operation duration	> 2 hours	> 1 hour	Exceeds
Functional test pass rate	100% (9/9)	100%	Meets

Obstacle Halt (48 ms $\pm$ 6 ms): Significantly better than comparable Arduino-based designs (60-80 ms, Chinmayi et al., 2018; $\sim$ 70 ms, Mothe et al., 2020), attributable to the ESP32's 240 MHz clock and direct GPIO ultrasonic polling with no serial overhead. The $\pm$ 6 ms SD reflects minor echo processing variation; at nominal speed the robot travels $\sim$ 3-4 mm within this window, well within the safe stopping margin (see Figure 1).

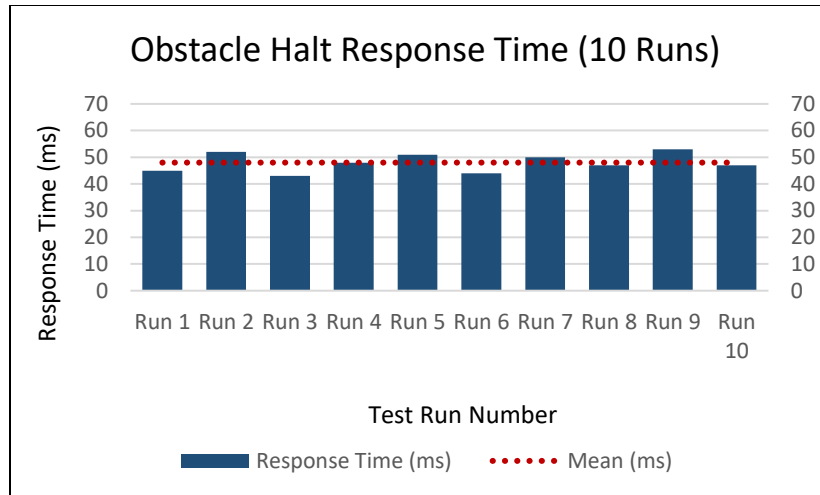


Figure 1: Obstacle Halt Response Time

Telegram Latency ($2.3 \text{ s} \pm 0.7 \text{ s}$): Composed of HTTPS handshake ($\sim 800 \text{ ms}$), payload transmission ($\sim 200 \text{ ms}$), and Telegram server push ($\sim 800\text{-}1200 \text{ ms}$ variable). All 15 events delivered with 100% message correctness - correct rack ID and accurate position description in every case (see Figure 2). The 5-second inter-alert cooldown prevents API rate-limiting under repeated detections.

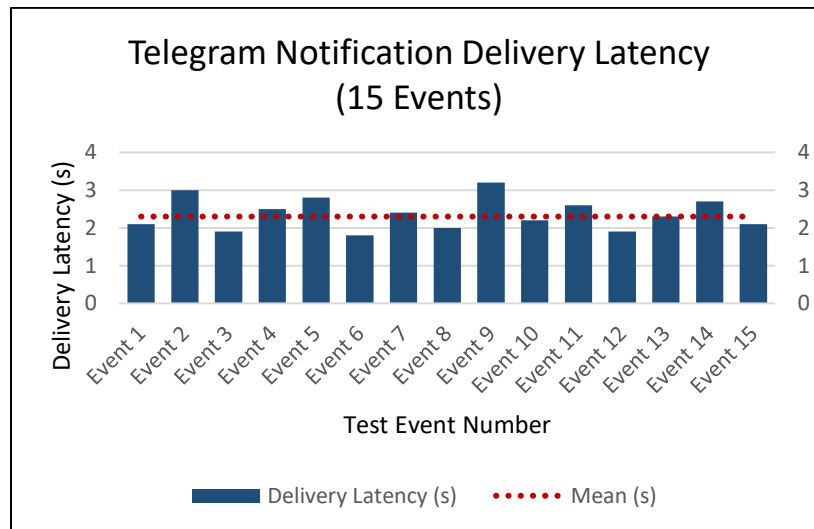


Figure 2: Telegram Notification Latency

RFID Miss Rate at Maximum Speed (6.7%): The PN532's I2C response window ($\sim 10\text{-}15 \text{ ms}$) is insufficient when the antenna clears a tag at maximum PWM before a read cycle completes. Operationally mitigated by reducing speed near known junction locations.

Line Recovery Limitation (80% at $> 45^\circ$): The fixed-increment sweep algorithm assumes departures within 45° , which covers wheel-slip and minor surface inconsistencies. Larger

departures exceed the maximum sweep range, causing recovery failure - the primary performance gap identified.

Bottleneck Assessment: No computational bottlenecks detected. Dual-core isolation was effective: Firebase write operations (50-200 ms) produced zero observable delay in obstacle halt response or IR loop frequency. The sole structural bottleneck is Wi-Fi dependency - if connectivity drops, Telegram and Firebase logging are lost, though autonomous navigation continues using locally cached RFID UIDs.

6. Performance Optimization and Recommendations

Optimizations Implemented: The dual Firebase object architecture (fbdo for reads, fbdo2 for writes) eliminated 50-200 ms blocking delays that previously interfered with RFID I2C polling when a single object handled concurrent operations. Non-blocking Telegram dispatch on Core 0 ensured motor halt latency was unaffected by network call duration. Local RFID UID caching at boot reduced junction processing time from ~150 ms to under 5 ms by eliminating per-junction Firebase queries. The 20 cm ultrasonic threshold was experimentally tuned down from 25 cm to eliminate floor-reflection false positives while maintaining adequate stopping distance.

Future Recommendations: Replace the fixed-increment RECOVERY sweep with an adaptive algorithm that dynamically increases sweep range, targeting 100% recovery at any departure angle. Implement a speed-reduction zone triggered by encoder count proximity to known junctions to resolve the RFID maximum-speed miss rate. Replace Firebase polling (currently every 300 ms for arm handshake) with Realtime Listeners to eliminate unnecessary reads under multi-robot operation. Adopt speed-compensated encoder distance computation using wheel circumference and pulses-per-revolution calibration to remove manual threshold recalibration when payload or speed changes.

7. Challenges and Solutions

I2C Timing Conflict: Default blocking PN532 I2C reads caused 15–30 ms navigation loop stalls. Resolved by switching to interrupt-driven mode (IRQ on GPIO 32), eliminating blocking waits entirely.

Power-On Motor Hazard: ESP32 GPIO float during boot caused brief unintended motor activation via L298N. Resolved by asserting all motor GPIO pins LOW explicitly at INIT entry before any sensor initialization, with a 2-second stabilization delay - eliminating the hazard across all TC7 trials.

Firebase Single-Object Blocking: Concurrent read/write on one Firebase object caused 50-200 ms stalls. Resolved with the dual-object architecture described in Section 6.

Ultrasonic False Positives: Floor surface irregularities at tape edges triggered false obstacle halts at the initial 25 cm threshold. Resolved by experimentally reducing threshold to 20 cm, which eliminated false positives while maintaining a sufficient stopping margin at nominal speed.

8. Ethical and Compliance Considerations

All testing was conducted within the SLIIT laboratory using purpose-built infrastructure with no real user data or personal information involved. The Firebase instance was a private project accessible only to the project team, authenticated via FIREBASE_AUTH token. Telegram messages were sent exclusively to a private bot chat accessible only to the student. No human participants were exposed to autonomous robot motion during data collection. All activities complied with SLIIT research project guidelines and applicable data privacy principles.

9. References

- [1] K. M. Hasan et al., "Sensor based autonomous color line follower robot with obstacle avoidance," *Proc. IEEE BEIAC*, pp. 598-603, 2013. doi: 10.1109/beiac.2013.6560199
- [2] L. J. Mohan and J. Ignatious, "Navigation of mobile robot in a warehouse environment," *Proc. IEEE ICETIETR*, pp. 1- 5, 2018. doi: 10.1109/icetietr.2018.8529012
- [3] R. Chinmayi et al., "Obstacle detection and avoidance robot," *Proc. IEEE ICCIC*, pp. 1- 6, 2018. doi: 10.1109/iccic.2018.8782344
- [4] R. Mothe et al., "An IoT based obstacle avoidance robot using ultrasonic sensor and Arduino," *IOP Conf. Ser. Mater. Sci. Eng.*, vol. 981, no. 4, 2020. doi: 10.1088/1757-899x/981/4/042002
- [5] NXP Semiconductors, "PN532 User Manual," Rev. 02, 2007. [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/pdf/595327/NXP/PN532.html>
- [6] Espressif Systems, "ESP32 Technical Reference Manual," 2023. [Online]. Available: https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf
- [7] Apache JMeter Documentation. [Online]. Available: <https://jmeter.apache.org>
- [8] Google Firebase Documentation - Realtime Database. [Online]. Available: <https://firebase.google.com/docs/database>